During the startup phase of Catalog Service, Spring Boot will read the variables defined in the surrounding environment, recognize that `POLAR_GREETING` can be mapped to the `polar.greeting` property, and store its value in the Spring `Environment` object, overriding the value defined in application.yml. The result will be the following:

```
$ http :9001/
Welcome to the catalog from ENV
```

After testing the application, stop the process with Ctrl-C. If you ran the application from Windows PowerShell, remember to unset the environment variable with `Remove-Item Env:\POLAR_GREETING`.

When you use environment variables for storing configuration data, you don't have to change the command to run your application (as you did for CLI arguments and JVM properties). Spring will automatically read environment variables from the context where it's deployed. This approach is less error-prone and fragile than using CLI arguments or JVM system properties.

NOTE You can use environment variables to define configuration values that depend on the infrastructure or platform where the application is deployed, such as profiles, port numbers, IP addresses, and URLs.

Environment variables work seamlessly on virtual machines, OCI containers, and Kubernetes clusters. However, they might not be enough. In the next section, I'll go through some of the issues affecting environment variables and how Spring Cloud Config can help address them.